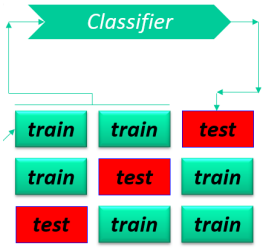
Lecture 3 - Model Validation

It is important to validate the performance of a classifier, in order to determine wether to employ the classifier or to improve the performance of the classifier.

k-Fold Cross-Validation

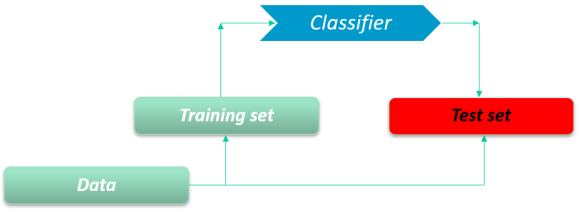
Split the data into *k* subsets of equal size (stratification). Each subset is in turn used for testing, and the rest is used for training (one model for each). It is most common to use 10-Fold cross validation. To further reduce variance you could also use repeated k-Fold cross validation.
A variation of this is **Leave-One-Out Cross-Validation**; *n*-Fold cross validation, where *n* is the total number of instances. This makes best use of the data and doesn't involve random sampling. Downsides are that it is computationally expensive, and there is no stratification.

- Large Data:
  Hold-Out
- Medium Data:
  k-Fold CV
- Small Data:
  Leave-One-Out



Hold-Out Method

Split (usually) 66% of data into a training set, and the remaining 34% into a test set. Only apply when you have enough data to do this meaningfully. The big advantage of the hold-out method is **stratification**; both the training- and the test set share the same distribution. The hold-out method is very unstable, it only starts to mean something if you do many trials and take the average. The main problem is that, even after repeating the method many times, there will be instances that are only ever used for training / testing, and not for both.



Metrics For Model Validation:

		Predicted Class: Positive	Predicted Class: Negative
Actual Class	P	True Positive	False Positive
	N	False Positive	True Negative

- Accuracy = (TP + TN) / (P + N)
- Error = (FP + FN) / (P + N)
- Precision = TP / (TP + FP)
- Recall / TP-Rate = TP / P
- FP-Rate = FP / N

Confidence Intervals

With N% probability, the actual error of classifier h lies within:

$$est_error(h) \pm z_N \sqrt{\frac{est_error(h)(1 - est_error(h))}{n}}$$

50%	68%	80%	90%	95%	98%	99%
0.67	1.00	1.28	1.64	1.96	2.33	2.58

Costs

Different types of classification errors incur different costs (e.g. missclassifying someone as a terrorist). Usually the *true positive cost* and the *true negative cost* are set to 0 (they are not errors).

Cost Sensitive Classification

Imagine a system with the following cost classification: *false positive cost = 10, false negative cost = 5*. It is known that 60% of the data is positive, and 40% is negative. The **expected cost** for classifying an instance as positive equals 0.6 x 0 + 0.4 x 10 = 4, and for classifying an instance as negative equals 0.6 x 5 + 0.4 x 0 = 3. To minimize the expected cost the instance is classified as negative.

Receiver Operating Characteristic (ROC)

In an ROC Space, the Y-axis represents the TP-Rate, and the X-axis represents the FP-Rate. Classifier A is **dominating** classifier B iff it is more top-left than classifier B in **both** directions (e.g. RED dominates GREEN in example).

True	Predicted: pos	Predicted: neg
pos	40	60
neg	30	70

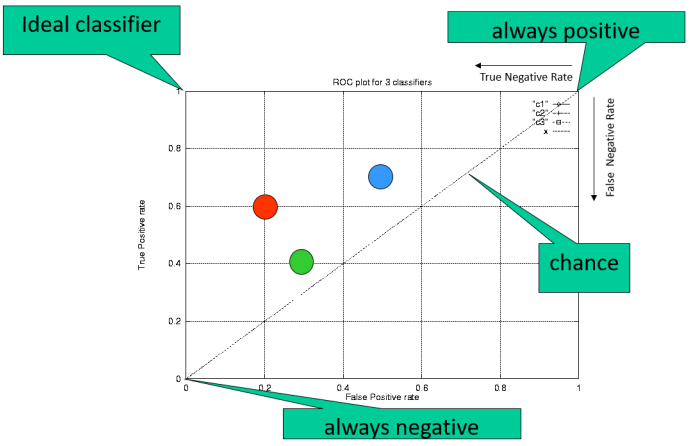
Classifier 1
TPr = 0.4
FPr = 0.3

True	Predicted: pos	Predicted: neg
pos	70	30
neg	50	50

Classifier 2
TPr = 0.7
FPr = 0.5

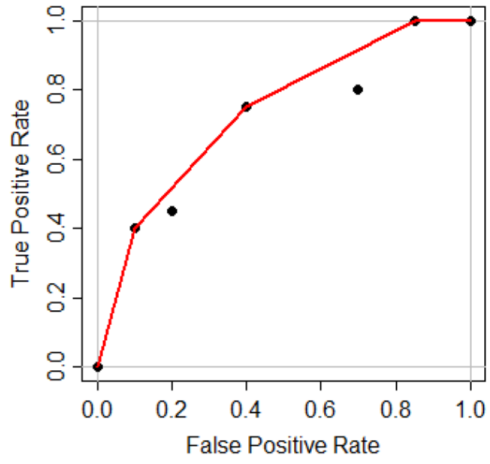
True	Predicted: pos	Predicted: neg
pos	60	40
neg	20	80

Classifier 3
TPr = 0.6
FPr = 0.2



ROC Convex Hull (ROCCH)

A line from (0, 0) to (1, 1) that contains all **optimal classifiers** (the best classifiers that don't dominate each other). Classifiers below this line are always sub-optimal. You can find the ROCCH by stretching a band from (0, 0) to (0, 1) to (1, 1), and then dropping it. The continuous line represents that a combination of two classifiers is also optimal.



ISO-Accuracy Line

Connect ROC points with the same accuracy, higher accuracy is always better. They have a slope of N/P. For uniform class distribution C4.5 is optimal and achieves 82% accuracy. With 4x as many positives as negatives, SVM is optimal and achieves 84% accuracy. With 4x as many negatives as positives, CN2 is optimal and achieves 86% accuracy. With less than 9% positives, always_negatives is optimal. With less than 11% negatives, always_positive is optimal.

Constructing ROC Curve

1) Sort the test instances into list *T* in decreasing order of the probability P_{pos} of being positively classified by classifier *h*.

2) For each instance *x* in list *T*:

Compute TP-Rate and FP-Rate of the classifier for this instance. Plot these rates onto the ROC Space